LAPORAN PRAKTIKUM

STRUKTUR DATA

ADT JAM

disusun Oleh:

RIFKI MAULANA

NIM 2511533007

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

ASISTEN PRAKTIKUM : ZAHRAN AZARIA ANVAYA

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 2 APRIL 2026

KATA PENGANTAR

Puji syukur kehadirat Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan kelima dengan judul "**ADT JAM**". Melalui penyusunan laporan ini, penulis berharap dapat memberikan gambaran mengenai cara pemrograman struktur data ADT JAM.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 2 April 2026

Penulis

DAFTAR ISI

LAPORAN PRAKTIKUM 1
KATA PENGANTAR i
DAFTAR ISI ii
DAFTAR GAMBAR iii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat Praktikum 1
BAB II PEMBAHASAN 2
2.1 Uraian kode 2
2.1.1 Class Jam 2
2.1.2 Class JamDriver_2511533007 3
2.1.3 Class JamDriver2_2511533007 3
2.2 Langkah Pengerjaan 5
2.2.1 Class Jam 5
2.2.2 Method validasi Jam 5
2.2.3 Konstruktor Jam 5
2.2.4 Selector Jam 5
2.2.5 Mutator Jam 6
2.2.6 Konversi dari detik ke detik Jam 6
2.2.7 Method relasional jam 6
2.2.8 Operasi Aritmatika 7
2.2.9 Method durasi 7
2.2.10 Method toString 7
2.2.11 Class JamDriver 7
2.2.12 Class JamDriver2 8
BAB III KESIMPULAN DAN SARAN 10
3.1 Kesimpulan 10
3.2 Saran 10
DAFTAR PUSTAKA 11

DAFTAR GAMBAR

gambar 1 5
gambar 2 5
gambar 3 5
gambar 4 6
gambar 5 6
gambar 6 6
gambar 7 7
gambar 8 7
gambar 9 7
gambar 10 7
gambar 11 8
gambar 12 8
gambar 13 9
gambar 14 9

BAB I
PENDAHULUAN

1.1 Latar Belakang

ADT adalah koleksi data dan operasi yang dapat digunakan untuk memanipulasi data tersebut. Class memiliki data dan fungsi. Data dan fungsi yang dideklarasi private tidak dapat diakses secara langsung oleh client, sementara data dan fungsi yang dideklarasi public akan bersifat publik dan dapat diakses oleh client secara langsung.

Pada praktikum ini, dibuat sebuah ADT Jam yang merepresentasikan waktu dalam format jam, menit, dan detik. ADT ini dirancang agar mampu melakukan berbagai operasi dasar seperti validasi waktu, konversi ke detik, perbandingan waktu, penjumlahan waktu, pengurangan waktu, serta pencarian waktu berikutnya atau sebelumnya.

1.2 Tujuan Praktikum

- Memahami konsep Abstract Data Type (ADT) dalam pemrograman.
- Mampu merancang dan mengimplementasikan ADT Jam menggunakan bahasa Java.
- Menerapkan konsep enkapsulasi melalui atribut, constructor, accessor, dan mutator.

1.3 Manfaat Praktikum

- Menambah pemahaman mengenai penerapan ADT dalam kasus kehidupan nyata.
- Melatih kemampuan membuat program yang terstruktur.
- Membantu memahami penggunaan class sebagai representasi objek di Java.

BAB II
PEMBAHASAN

2.1 Uraian kode

2.1.1 Class Jam

```
package pekan1_2511533007;

public final class Jam_2511533007 {
      private int hh; // 0..23
      private int mm; // 0..59
      private int ss; // 0..59

      // --- Validator ---
    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
    }

    // --- Konstruktor/Kreator ---
    public Jam_2511533007(int h, int m, int s) {
        this.hh = h; this.mm = m; this.ss = s;
    }

    // --- Selektor ---
    public int getHour() { return hh; }
    public int getMinute() { return mm; }
    public int getSecond() { return ss; }

    // --- Mutator ---
    public void setHour(int h) { this.hh = h; }
    public void setMinute(int m) { this.mm = m; }
    public void setSecond(int s) { this.ss = s; }

    // --- Konversi ---
    public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
    public static Jam_2511533007 fromSeconds(int total) {
       if (total < 0 ) throw new IllegalArgumentException("detik negatif");
       total %= 24 * 3600; //wrap 24 jam
       int h = total / 3600; total %= 3600;
       int m = total / 60; int s = total % 60;
       return new Jam_2511533007(h, m, s);
    }
    // --- Relasiomal ---
    public int compareTo(Jam_2511533007 other) { return
Integer.compare(this.toSeconds(), other.toSeconds()); }
    public boolean equals(Object o) {
       if (!(o instanceof Jam_2511533007 j)) return false;
       return hh == j.hh && mm == j.mm && ss == j.ss;
    }
    public int hashCode() {return java.util.Objects.hash (hh, mm, ss); }

    // --- Aritmatika ---
    public Jam_2511533007 plus(Jam_2511533007 other) {return
fromSeconds(this.toSeconds() + other.toSeconds()); }
    public Jam_2511533007 minus(Jam_2511533007 other) {return
fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600)); }
```

```
    public Jam_2511533007 nextSecond() { return fromSeconds(this.toSeconds() + 1);
}
    public Jam_2511533007 nextNSeconds(int n) { return
fromSeconds(this.toSeconds() + Math.max(0, n)); }
    public Jam_2511533007 prevSecond() { return
fromSeconds(Math.floorMod(this.toSeconds() - 1, 24 * 3600)); }
    public Jam_2511533007 prevNSeconds(int n) {return
fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24 * 3600)); }

    // --- Durasi (detik) --- bisa negatif jika this > other? sesuai spesifikasi
durasi (JAw, JAkh)
    public static int durasiDetik(Jam_2511533007 jaw, Jam_2511533007 jakh) {
return jakh.toSeconds() - jaw.toSeconds(); }
    // --- Overriding ke string ---
    public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss);
}


}
```

2.1.2 Class JamDriver_2511533007

```
package pekan1_2511533007;

public class JamDriver_2511533007 {

      public static void main(String[] args) {
            Jam_2511533007 a = new Jam_2511533007(23, 59, 50);
            Jam_2511533007 b = new Jam_2511533007(0, 0, 15);
            System.out.println("a                          = " + a);
            System.out.println("b                          = " + b);
            System.out.println("a+b                        = " + a.plus(b));
            System.out.println("next 20s            = " + a.nextNSeconds(20));
            System.out.println("durasi(a,b)      = " +
Jam_2511533007.durasiDetik(a, b));
            System.out.println("a.compareTo(b)  = " + a.compareTo(b));
      }

}
```

2.1.3 Class JamDriver2_2511533007

```
package pekan1_2511533007;
import java.util.Scanner;
public class JamDriver2_2511533007 {

      public static void main(String[] args) {
            Scanner input = new Scanner(System.in);
            System.out.println("=== Program driver Objek Jam ===");
            // 1. Input jam Pertama
            System.out.println("\n--- Input jam 1 ---");
            Jam_2511533007 j1 = buatJamDariInput(input);

            // 2. Input Jam Kedua
            System.out.println("\n --- Input Jam 2 ---");
            Jam_2511533007 j2 = buatJamDariInput(input);

            // 3. Menampilkan Data
            System.out.println("\n--- Hasil Operasi ---");
            System.out.println("Jam 1 (String)                    : " +
j1.toString());
```

```
            System.out.println("Jam 2 (String                    : " +
j2.toString());
            System.out.println("Jam 1 dalam detik         : " + j1.toSeconds());
            System.out.println("Jam 2 dalam detik         : " + j2.toSeconds());

            // 4. Operasi Relational (perbandingan)
            int perbandingan = j1.compareTo(j2);
            if (perbandingan > 0 ) {
                  System.out.println("Status                    : jam 1 lebih
lambat (setelah) Jam 2");
            } else if (perbandingan < 0) {
                  System.out.println("Status                    : Jam 1 lebih
awal (sebelum) Jam 2");
            }else {
                  System.out.println("Status                          : Jam 1
dan Jam 2 sama persis");
            }

            // 5. Operasi Aritmatika
            System.out.println("Durasi (J1 ke J2)       : " +
Jam_2511533007.durasiDetik(j1, j2) + " detik" );

            Jam_2511533007 jNext = j1.nextSecond();
          System.out.println("Jam 1 Detik Berikutnya: " + jNext);

          Jam_2511533007 jPrev = j1.prevSecond();
          System.out.println("Jam 1 Detik Sebelumnya: " + jPrev);

          // 6. Operasi Penjumlahan Jam
          Jam_2511533007 jHasilPlus = j1.plus(j2);
          System.out.println("Hasil J1 + J2           : " + jHasilPlus);

          input.close();
          System.out.println("\nProgram Selesai.");
}
    /**
     * Prosedur pembantu untuk melakukan input dan validasi secara berulang
     * sampai user memasukkan angka yang benar (0..23, 0..59).
     */
    private static Jam_2511533007 buatJamDariInput(Scanner sc) {
        int h, m, s;
        while (true) {
            System.out.print("Masukkan Jam (0-23)   : ");
            h = sc.nextInt();
            System.out.print("Masukkan Menit (0-59) : ");
            m = sc.nextInt();
            System.out.print("Masukkan Detik (0-59) : ");
            s = sc.nextInt();

            // Memanggil method static isValid dari kelas Jam
            if (Jam_2511533007.isValid(h, m, s)) {
                return new Jam_2511533007(h, m, s);
            } else {
                System.out.println("[Error] Input tidak valid! Silakan
ulangi.\n");
            }
        }
    }
```

```
}
```

2.2 Langkah Pengerjaan

2.2.1 Class Jam

Merupakan inti dari program karena berufungsi sebagai ADT Jam, dikelas ini terdapat 3 atribut yaitu hh (sebagai jam), mm(sebagai menit), dan ss (sebagai ss).

```
public final class Jam_2511533007 {
    private int hh; // 0..23
    private int mm; // 0..59
    private int ss; // 0..59
```

gambar 1

2.2.2 Method validasi Jam

untuk memeriksa apakah input waktu yang dimasukkan berada dalam batas yang benar.

```
// --- Validator ---
public static boolean isValid(int h, int m, int s) {
    return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
}
```

gambar 2

2.2.3 Konstruktor Jam

Constructor digunakan untuk membuat objek Jam baru dengan nilai jam, menit, dan detik tertentu.

```
// --- Konstruktor/Kreator ---
public Jam_2511533007(int h, int m, int s) {
    this.hh = h; this.mm = m; this.ss = s;
}
```

gambar 3

2.2.4 Selector Jam

Method berikut digunakan untuk mengambil nilai atribut hour, minute, dan second. Ketiga method ini merupakan bentuk enkapsulasi, karena atribut tidak diakses langsung dari luar class, melainkan melalui method getter.

```
// --- Selektor ---
public int getHour() { return hh; }
public int getMinute() { return mm; }
public int getSecond() { return ss; }
```

gambar 4

2.2.5 Mutator Jam

Method ini digunakan untuk mengubah nilai jam, menit, dan detik pada objek. Sama seperti constructor

```
// --- Mutator ---
public void setHour(int h) { this.hh = h; }
public void setMinute(int m) { this.mm = m; }
public void setSecond(int s) { this.ss = s; }
```

gambar 5

2.2.6 Konversi dari detik ke detik Jam

a. Method toSecond

Method ini mengubah waktu dalam format jam-menit-detik menjadi total detik.

b. Method fromSecond

Method ini mengubah total detik menjadi objek Jam kembali. Jika nilai detik lebih dari 24 jam, maka program menggunakan operasi modulo agar waktu kembali ke rentang satu hari penuh.

```
// --- Konversi ---
public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
public static Jam_2511533007 fromSeconds(int total) {
    if (total < 0 ) throw new IllegalArgumentException("detik negatif");
    total %= 24 * 3600; //wrap 24 jam
    int h = total / 3600; total %= 3600;
    int m = total / 60; int s = total % 60;
    return new Jam_2511533007(h, m, s);
```

gambar 6

2.2.7 Method relasional jam

a. CompareTo(Jam_2511533007)

Method ini membandingkan dua objek Jam berdasarkan total detik

b. Equals(Object o)

Method ini digunakan untuk mengecek kesamaan dua objek Jam berdasarkan isi atributnya

c. hashCode()

Method ini digunakan untuk menghasilkan nilai hash yang konsisten dengan equals()

```
// --- Relasiomal ---
public int compareTo(Jam_2511533007 other) { return Integer.compare(this.toSeconds(), other.toSeconds()); }
public boolean equals(Object o) {
    if (!(o instanceof Jam_2511533007 j)) return false;
    return hh == j.hh && mm == j.mm && ss == j.ss;
}
public int hashCode() {return java.util.Objects.hash (hh, mm, ss); }
```

gambar 7

2.2.8 Operasi Aritmatika

Berisi operasi aritmatika seperti tambah, kurang, lanjut satu detik setelah derik sekarang, lanjut satu detik setelah n detik, waktu satu detik sebelumnya, dan waktu n detik sebelumnya

```
// --- Aritmatika ---
public Jam_2511533007 plus(Jam_2511533007 other) {return fromSeconds(this.toSeconds() + other.toSeconds()); }
public Jam_2511533007 minus(Jam_2511533007 other) {return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600)); }
public Jam_2511533007 nextSecond() { return fromSeconds(this.toSeconds() + 1); }
public Jam_2511533007 nextNSeconds(int n) { return fromSeconds(this.toSeconds() + Math.max(0, n)); }
public Jam_2511533007 prevSecond() { return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24 * 3600)); }
public Jam_2511533007 prevNSeconds(int n) {return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24 * 3600)); }
```

gambar 8

2.2.9 Method durasi

Method ini menghitung selisih waktu dalam satuan detik antara waktu awal dan waktu akhir.

```
// --- Durasi (detik) --- bisa negatif jika this > other? sesuai spesifikasi durasi (JAw, JAkh)
public static int durasiDetik(Jam_2511533007 jaw, Jam_2511533007 jakh) { return jakh.toSeconds() - jaw.toSeconds(); }
```

gambar 9

2.2.10 Method toString

Method ini mengubah objek Jam menjadi string dengan format hh:mm:ss

```
// --- Overriding ke string ---
public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss); }
```

gambar 10

2.2.11 Class JamDriver

Kelas ini berfungsi sebagai driver atau penguji program. Driver ini digunakan untuk membuktikan semua method yang ada pada class Jam berjalan

```java
package pekan1_2511533007;

public class JamDriver_2511533007 {

    public static void main(String[] args) {
        Jam_2511533007 a = new Jam_2511533007(23, 59, 50);
        Jam_2511533007 b = new Jam_2511533007(0, 0, 15);
        System.out.println("a                 = " + a);
        System.out.println("b                 = " + b);
        System.out.println("a+b               = " + a.plus(b));
        System.out.println("next 20s          = " + a.nextNSeconds(20));
        System.out.println("durasi(a,b)       = " + Jam_2511533007.durasiDetik(a, b));
        System.out.println("a.compareTo(b)    = " + a.compareTo(b));
    }

}
```

gambar 11

2.2.12 Class JamDriver2

Kelas ini merupakan driver yang menerima input dari user. Logicnya yaitu:

a. Program meminta input jam, menit, dan detik untuk jam pertama
b. Program meminta input jam, menit, dan detik untuk jam kedua
c. Data divalidasi menggunaka isValid()
d. Jika input user valid maka objerk jam dibuat
e. Kemudian program akan menampilkan operasi aritmatika

```java
package pekan1_2511533007;
import java.util.Scanner;
public class JamDriver2_2511533007 {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("=== Program driver Objek Jam ===");
        // 1. Input jam Pertama
        System.out.println("\n--- Input jam 1 ---");
        Jam_2511533007 j1 = buatJamDariInput(input);

        // 2. Input Jam Kedua
        System.out.println("\n --- Input Jam 2 ---");
        Jam_2511533007 j2 = buatJamDariInput(input);

        // 3. Menampilkan Data
        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (String)          : " + j1.toString());
        System.out.println("Jam 2 (String           : " + j2.toString());
        System.out.println("Jam 1 dalam detik       : " + j1.toSeconds());
        System.out.println("Jam 2 dalam detik       : " + j2.toSeconds());

        // 4. Operasi Relational (perbandingan)
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0 ) {
            System.out.println("Status                  : jam 1 lebih lambat (setelah) Jam 2");
        } else if (perbandingan < 0) {
```

gambar 12

```java
        System.out.println("Status              : Jam 1 lebih awal (sebelum) Jam 2");
    }else {
        System.out.println("Status              : Jam 1 dan Jam 2 sama persis");
    }

    // 5. Operasi Aritmatika
    System.out.println("Durasi (J1 ke J2)       : " + Jam_2511533007.durasiDetik(j1, j2) + " detik" );

    Jam_2511533007 jNext = j1.nextSecond();
    System.out.println("Jam 1 Detik Berikutnya: " + jNext);

    Jam_2511533007 jPrev = j1.prevSecond();
    System.out.println("Jam 1 Detik Sebelumnya: " + jPrev);

    // 6. Operasi Penjumlahan Jam
    Jam_2511533007 jHasilPlus = j1.plus(j2);
    System.out.println("Hasil J1 + J2           : " + jHasilPlus);

    input.close();
    System.out.println("\nProgram Selesai.");

/**
 * Prosedur pembantu untuk melakukan input dan validasi secara berulang
 * sampai user memasukkan angka yang benar (0..23, 0..59).
 */
private static Jam_2511533007 buatJamDariInput(Scanner sc) {
    int h, m, s;
```

gambar 13

```java
    private static Jam_2511533007 buatJamDariInput(Scanner sc) {
        int h, m, s;
        while (true) {
            System.out.print("Masukkan Jam (0-23)   : ");
            h = sc.nextInt();
            System.out.print("Masukkan Menit (0-59) : ");
            m = sc.nextInt();
            System.out.print("Masukkan Detik (0-59) : ");
            s = sc.nextInt();

            // Memanggil method static isValid dari kelas Jam
            if (Jam_2511533007.isValid(h, m, s)) {
                return new Jam_2511533007(h, m, s);
            } else {
                System.out.println("[Error] Input tidak valid! Silakan ulangi.\n");
            }
        }
    }

}
```

gambar 14

BAB III
KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa ADT Jam dapat diimplementasikan dengan baik menggunakan class Java. Objek Jam berhasil merepresentasikan waktu dalam format jam, menit, dan detik, serta memiliki berbagai operasi yang mendukung manipulasi waktu.

Program ini menunjukkan penerapan konsep ADT melalui validasi data, konversi satuan waktu, perbandingan objek, dan operasi aritmatika. Selain itu, penggunaan driver program membantu membuktikan bahwa class Jam bekerja sesuai dengan spesifikasi yang diharapkan.

3.2 Saran

a. Penamaan method dan komentar sebaiknya dibuat lebih konsisten agar kode lebih mudah dibaca dan dipelajari.
b. Penanganan error pada input pengguna dapat dibuat lebih baik, misalnya dengan menangani input bukan angka.

DAFTAR PUSTAKA

[1] R. Muliono, “ADT (Abstract Data Type),” [Daring]. Tersedia pada: https://rizkimuliono.blog.uma.ac.id/wp-content/uploads/sites/365/2017/05/ADT-Abstract-Data-Type.pdf. [Diakses: 02-Apr-2026].

[2] W3Schools, “Java Constructors,” [Daring]. Tersedia pada: https://w3schoolsua.github.io/java/java_constructors_en.html#gsc.tab=0. [Diakses: 02-Apr-2026].

Laporan Tugas

a. Kode program

```
package tugasADT;

import java.util.ArrayList;

public class ADTmobil_2511533007 {
    private String nama;
    private int tahun;
    private int cc;
    private int harga;
    private String merk;
    private ArrayList<ADTmobil_2511533007> daftarMobil;

    public ADTmobil_2511533007() {
        this.daftarMobil = new ArrayList<>();
    }

    public ADTmobil_2511533007(String nama, int tahun, int cc, int harga, String
merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }

    public String getNama() {
       return nama;
    }
    public int getTahun() {
       return tahun;
    }
    public int getCc() {
       return cc;
    }
    public int getHarga() {
       return harga;
    }
    public String getMerk() {
       return merk;
    }

    public ArrayList<ADTmobil_2511533007> getDaftarMobil() {
       return daftarMobil;
    }

    public void setNama(String nama) {
       this.nama = nama;
    }
    public void setTahun(int tahun) {
       this.tahun = tahun;
    }
    public void setCc(int cc) {
       this.cc = cc;
    }
    public void setHarga(int harga) {
```

```
        this.harga = harga;
    }
    public void setMerk(String merk) {
        this.merk = merk;
    }

    public void tambahMobil(ADTmobil_2511533007 m) {
        daftarMobil.add(m);
        System.out.println("Daftar mobil : " + this.getDaftarMobil());
    }

    public void hapusMobil(ADTmobil_2511533007 m) {
        daftarMobil.remove(m);
        System.out.println("Daftar mobil setelah dihapus : " +
this.getDaftarMobil());
    }

    public String toString() {
        return nama + ", " + tahun + ", " +cc + ", " + harga + ", " + merk;
    }
}
```

```
package tugasADT;

public class MobilDriver_2511533007 {
    public static void main(String[] args) {
        ADTmobil_2511533007 daftar = new ADTmobil_2511533007();

        ADTmobil_2511533007 m1 = new ADTmobil_2511533007("Avanza", 2020, 1300,
150000000, "Toyota");
        ADTmobil_2511533007 m2 = new ADTmobil_2511533007("Xpander", 2021, 999,
180000000, "Mitsubishi");

        daftar.tambahMobil(m1);
        daftar.tambahMobil(m2);

        daftar.hapusMobil(m1);
    }
}
```

b. Penjelasan kode program

Kode tersebut membuat ADT sederhana untuk data mobil dan sebuah program driver untuk mencoba operasinya.

ADTmobil_2511533007 menyimpan atribut mobil: nama, tahun, cc, harga, dan merk. Selain itu, kelas ini juga punya ArrayList<ADTmobil_2511533007> bernama daftarMobil untuk menampung data mobil. Di dalamnya ada getter, setter, tambahMobil() untuk menambahkan mobil ke daftar, hapusMobil() untuk menghapus mobil dari daftar, dan toString() agar isi objek bisa ditampilkan dalam bentuk teks.

MobilDriver_2511533007 adalah kelas utama yang dijalankan. Di sini dibuat objek daftar mobil, lalu dibuat dua data mobil: Avanza dan Xpander. Setelah itu, kedua mobil dimasukkan ke daftar dengan tambahMobil(), lalu mobil Avanza dihapus dengan hapusMobil().